

1. Contextualização

Uma empresa especializada na organização de eventos necessita de um sistema para controlar seus eventos, permitindo o cadastro, consulta, alteração e exclusão de informações. Além do gerenciamento dos dados armazenados em banco de dados, a empresa deseja disponibilizar as informações por meio de uma interface Web e utilizar dados provenientes de uma API externa.

Os alunos deverão desenvolver o EventTech — Sistema de Gestão de Eventos, aplicando, de forma integrada, os conceitos estudados ao longo das 12 aulas. O projeto deverá demonstrar funcionamento, integração, testes, organização e domínio técnico.

2. Objetivo do projeto

- cadastrar, consultar, alterar e excluir eventos;
- armazenar dados em Oracle;
- utilizar Programação Orientada a Objetos;
- disponibilizar dados por HTML e CSS;
- receber dados por formulários HTML;
- consumir uma API REST externa;
- trabalhar com JSON;
- disponibilizar dados do sistema em JSON;
- tratar erros e validar entradas;
- organizar o projeto em arquivos e módulos;
- demonstrar testes de funcionamento.

3. Tema obrigatório

O projeto deverá trabalhar exclusivamente com EVENTOS. Exemplos: palestras, workshops, congressos, feiras, seminários, cursos, eventos corporativos, culturais ou tecnológicos. A equipe poderá escolher o segmento, mantendo o foco no gerenciamento de eventos.

4. Conceitos obrigatórios

Conceito	Obrigatório
Python	SIM
Funções	SIM
POO / classes / objetos	NÃO
Oracle / SQL	SIM
CRUD completo	SIM
INSERT / SELECT / UPDATE / DELETE	SIM
Bind variables	SIM
commit() / rollback()	SIM
Exceções	SIM
API REST / HTTP / GET	SIM
Parâmetros / requests	SIM
JSON	SIM

Flask / rotas	SIM
HTML / CSS / Jinja	SIM
Formulários Web	SIM
Flask + Oracle	SIM
Exportação JSON	SIM
Organização modular	SIM
Testes	SIM

5. Banco de dados

Utilize a tabela abaixo como estrutura mínima:

```
CREATE TABLE EVENTOS (
  ID_EVENTO NUMBER PRIMARY KEY,
  NOME_EVENTO VARCHAR2(100) NOT NULL,
  CATEGORIA VARCHAR2(50) NOT NULL,
  DATA_EVENTO DATE NOT NULL,
  LOCAL_EVENTO VARCHAR2(100) NOT NULL,
  CAPACIDADE NUMBER NOT NULL,
  VALOR_INGRESSO NUMBER(10,2),
  RESPONSAVEL VARCHAR2(100) NOT NULL,
  STATUS VARCHAR2(20) DEFAULT 'ATIVO',
  DATA_CADASTRO DATE DEFAULT SYSDATE
);
```

A equipe poderá acrescentar campos, desde que consiga justificar tecnicamente sua necessidade.

6. Programação Orientada a Objetos

O projeto deverá possuir uma classe responsável pela representação do evento e essa classe deverá ser efetivamente utilizada.

class Evento:

```
def __init__(self, id_evento, nome_evento, categoria, data_evento,
             local_evento, capacidade, valor_ingresso,
             responsavel, status):
    self.id_evento = id_evento
    self.nome_evento = nome_evento
    self.categoria = categoria
    self.data_evento = data_evento
    self.local_evento = local_evento
    self.capacidade = capacidade
    self.valor_ingresso = valor_ingresso
    self.responsavel = responsavel
    self.status = status
```

7. CRUD obrigatório

CREATE

Cadastro por formulário, validação, criação do objeto, INSERT, bind variables, commit() e tratamento de erros.

READ

Listagem de todos os eventos e consulta individual por ID.

UPDATE

Verificação de existência, edição, UPDATE, bind variables, commit() e tratamento de erros.

DELETE

Identificação, confirmação, DELETE, tratamento de inexistência e commit()/rollback().

8. API REST

Consuma uma API REST externa. A equipe deverá identificar a API, endpoint, método HTTP, parâmetros, retorno JSON e tratamento de erros. A API deverá ter relação funcional com o sistema; não basta imprimir o resultado no terminal.

Exemplos: localização, endereço, clima, feriados, países, cidades ou dados geográficos relacionados ao evento.

9. JSON

O sistema deverá trabalhar com JSON recebido da API externa e produzir JSON próprio contendo os eventos cadastrados. O JSON poderá ser disponibilizado por arquivo, rota Flask ou ambos.

10. Flask, HTML e CSS

O sistema deverá ser acessado pelo navegador e utilizar Flask. Rotas sugeridas: /, /eventos, /eventos/cadastrar, /eventos/editar/<id>, /eventos/excluir/<id>, /eventos/<id> e /api/eventos.

Deverá possuir, no mínimo, página inicial, página de eventos, cadastro, alteração e consulta.

11. Validações e erros

- Nome e categoria não podem ficar vazios.
- Data deve ser válida.
- Capacidade deve ser maior que zero.
- Valor não pode ser negativo.
- Responsável não pode ficar vazio.
- ID não pode ser duplicado.
- Tratar banco indisponível, conexão, ID inexistente, formulário incompleto, valores inválidos, API indisponível, timeout, resposta HTTP inesperada e JSON inválido.

- Não exibir traceback diretamente ao usuário.

12. Regras de desenvolvimento

1. O código deverá ser autoral e desenvolvido pela própria equipe.
2. Todos os integrantes deverão compreender o código entregue.
3. O professor poderá solicitar explicações, alterações ou demonstrações durante a apresentação.
4. Não será aceito código que nenhum integrante consiga explicar.
5. A nota do projeto é coletiva, mas o domínio técnico é individual.

13. Uso de Inteligência Artificial

A IA não é proibida, mas será regulamentada. Ela poderá ser utilizada como ferramenta de aprendizagem, não como substituta do desenvolvimento.

Permitido

- explicar conceitos e erros
- sugerir formas de estudo
- revisar conceitos
- sugerir melhorias
- explicar documentação
- ajudar a identificar erros
- sugerir casos de teste
- discutir alternativas de implementação

Permitido com registro

Pequenos exemplos, explicações de sintaxe, exemplos de bibliotecas e comparações entre alternativas, desde que o resultado seja compreendido e adaptado pela equipe.

Não permitido

- gerar o projeto inteiro
- gerar o CRUD inteiro
- gerar todas as classes
- gerar o sistema Flask completo
- gerar o projeto a partir do enunciado
- copiar código sem compreender
- solicitar que a IA faça a atividade integralmente
- entregar código que nenhum integrante consegue explicar

IA pode ser ferramenta de aprendizagem; não pode substituir o desenvolvimento, o raciocínio ou a autoria do aluno.

14. Registro obrigatório de IA

Se a equipe utilizar IA, deverá entregar REGISTRO_IA.md contendo:

6. Ferramenta utilizada
7. Data
8. Objetivo da utilização
9. Prompt utilizado
10. Resposta obtida
11. O que foi aproveitado
12. O que foi alterado pela equipe
13. Quem compreendeu/testou o resultado

15. README obrigatório

- Nome do projeto
- Integrantes e RM
- Objetivo
- Tecnologias
- Como executar
- Script SQL
- API utilizada
- Estrutura do projeto
- Uso de IA

16. Entregáveis

- Código-fonte
- Script SQL
- README.md
- REGISTRO_IA.md
- eventtech.json
- Evidências de testes